

Recod.ai/LUC - Scientific Image Forgery Detection

Max Bakke Seymour¹, Einar Martin Søvik Bernsen¹, and Emre Balci¹

¹University of Bergen (UiB)

Abstract

TODO - "Short summary of the paper, including results. Often written last".

1 Introduction

1.1 Copy-Move Forgery Detection

Copy-Move Forgery is a class of image forgery where regions of an image are duplicated and moved to other areas of the image. Commonly, techniques such as rotation, flipping and scaling of the copied region are applied, as well as post-processing such as edge smoothing, brightness adjustment and adding extra noise. These techniques can lead to forgeries which are hard to detect both through manual review and automated detectors. This method of forgery can be used within scientific images in order to fabricate results, where the complexity of the figures creates additional difficulties. [1]

1.2 Related works

TODO: What have others tried to solve this problem? Are there similar problems studied?

Copy-move image forgery detection has been studied through the lens of many different techniques. Early approaches focused on hand-crafted descriptors extracted either densely over overlapping blocks or sparsely at keypoints, followed by nearest-neighbour matching and geometric consistency checks.

1.3 Objectives

TODO

We aimed to create a full pipeline for CMFD which could perform well on the Recoid.ai Scientific Image Forgery Detection dataset. Ideally, our model should be robust towards post-processing techniques. The main goal is to perform well on the competition metric, which is a modified version of F1 which computes pixelwise F1 on pairs of (*predicted mask*, *ground truth mask*) using the following formula [2]:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

This calculation is applied to every connected component of forged pixels in the image (belonging to either the source or target). Predictions given by the model are matched up with the best corresponding ground-truth areas using the Hungarian algorithm. For any additional predicted areas predicted by the model above the actual amount of ground-truth components, a penalty is computed:

$$P = \frac{n_{gt}}{\max(n_{pred}, n_{gt})}$$

Finally, the per-image score is given as:

$$oF1 = \text{mean}(\text{best matched pairwise F1 scores}) \times P$$

The competition metric is challenging for a few reasons. Firstly, it requires high pixel-level precision for the output masks. In addition, a score of 0 for a given image is given if any pixels are predicted as forged on a ground-truth authentic image. Finally, it is highly punishing if any additional components are predicted, even if they should be small, as this affects the penalty P .

1.4 Contributions

TODO: What can the reader learn from this paper?

This paper presents a wide range of techniques for Copy-Move forgery and discusses strengths and weaknesses of the different techniques, particularly in relation to Copy-Move forgery on scientific images.

A number of different methods and architectures are tested in attempt to give a more comprehensive picture and explain in what circumstances different techniques may be useful.

2 Methods

TODO: What did we do? Reader should be able to reproduce results.

Basically, detailed breakdown of final implementation goes here.

Our initial implementation was inspired by a notebook by Kaggle user Gaurav Parkhedkar [3], who successfully employed a range of different techniques to perform a score of 0.332 on the competition metric on the test set.

2.1 Dataset

Unfortunately, we were unable to test our implementation against the Kaggle test-set, as we were too late to enter the competition. As we did not have access to the test data, we instead chose to use the available training data in a train (80%), evaluate (10%), test (10%) split. The training data consists of a base 2751 forged and 2377 authentic images as well as an additional 48 forged images.

3 Results

TODO: Show results of experiments. NB: Without interpretation at this stage! Align with method section.

4 Discussion

TODO: Interpret results Relate to objectives (did we achieve them?) Compare against others

4.1 Mask post-processing

4.2 Deep Cross-Scale PatchMatch

Inspired by the paper, we used a slightly larger dataset to train this model, inspired by the author’s use of a large self-generated copy-move dataset. The hope was that this simpler dataset would be helpful in allowing the model to generalize.

For this implementation

We assess that several factors contributed to the worse performance of this architecture. Firstly, it was discovered after training was terminated that the images often contain the same object copied multiple times (TODO: EXAMPLE). As each pixel only keeps it’s most similar offset, this means that any object which is copied more than once risks unstable performance within the PatchMatch module. For instance, let the source region A have two copied target regions A' and A'' within the image. The PatchMatch algorithm may match some pixels from A to A' , while other pixels may be matched to their corresponding pixel within A'' . Then, the generated offsets for A are less consistent, causing high DLF errors and in turn giving a poor signal to the decoder.

A better implementation for future work should involve some remedy for this, such as keeping the top- k matches per pixel instead of only one. In contrast, this is not an issue for purely feature-extraction based approaches such as our main pipeline.

4.3 Comments on kaggle submissions

At the time of writing, the maximum oF1 score achieved on hold-out test data was 0.458, achieved

by Kaggle user *orshanec*. [1]

References

- [1] Kaggle. *Recod.ai/LUC - Scientific Image Forgery Detection*. <https://www.kaggle.com/competitions/recodai-luc-scientific-image-forgery-detection/overview>. [Online; last accessed April-2026]. 2025.
- [2] Kaggle. *RecodAI F1*. <https://www.kaggle.com/code/metric/recodai-f1/>. [Online; accessed January-2026]. 2025.
- [3] G. Parkhedkar. *DINOv2 Base [0.332] — High-Res 4500px — Robust Inf*. <https://www.kaggle.com/code/gauravparkhedkar/dinov2-base-0-332-high-res-4500px-robust-inf>. [Online; accessed January-2026]. 2026.

A Appendix Example